Milestone – 3 Report

Exoplanet Habitability Prediction using Machine Learning

1. Introduction

This project focuses on predicting whether an exoplanet is habitable or not using a trained machine learning model. The system is developed using a Flask backend and a simple web-based frontend that allows users to input planetary parameters and receive predictions.

The project is divided into two major parts:

- Backend (Flask + Machine Learning Model)
- Frontend (HTML, CSS for user interaction)

2. Objectives

- To build a machine learning model that predicts exoplanet habitability.
- To deploy the model using Flask as a web API.
- To design a user-friendly frontend for entering planet data and displaying predictions.

3. Technologies Used

Backend

- Python
- Flask
- Joblib
- NumPy
- Pandas

Frontend

- HTML
- CSS
- JavaScript (optional)

4. Backend Implementation

4.1 Flask Framework

Flask is a lightweight web framework used to build web applications and APIs in Python. In this project, Flask is used to expose the machine learning model as a REST API.

4.2 Loading the Machine Learning Model

The trained Random Forest model is saved as a .pkl file and loaded using the Joblib library when the Flask application starts.

4.3 API Routes

Home Route (/)

This route loads the main web page of the application.

Prediction Route (/predict)

This route receives input values from the frontend, processes them, and sends them to the machine learning model. The model then returns a prediction indicating whether the planet is habitable.

4.4 Data Processing

The input data from the user is converted into numerical format and arranged into a Pandas DataFrame before being passed to the model for prediction.

5. Frontend Implementation

5.1 User Interface Design

The frontend is designed using HTML and CSS to provide a simple and interactive form for entering planetary parameters.

5.2 Input Fields

The form includes input fields for parameters such as:

- Planet radius
- Orbital period
- Equilibrium temperature
- Stellar flux

5.3 Communication with Backend

When the user submits the form, the data is sent to the Flask backend using an HTTP POST request. The backend processes the data and returns the prediction, which is displayed on the webpage.

6. Project Workflow

1. User opens the web application.
2. User enters exoplanet parameters in the form.
3. Frontend sends data to Flask backend.
4. Backend processes data and calls the machine learning model.
5. Model generates prediction.
6. Prediction is sent back and displayed to the user.

7. Folder Structure

```
project/
│
├── backend/
│   └── app.py
│
├── frontend/
│   └── templates/
│       └── index.html
│
├── models/
│   └── random_forest.pkl
│
└── notebooks/
```

└── Milestone-3.pdf

8. Results

The system successfully predicts whether an exoplanet is habitable based on the input parameters. The integration of machine learning with Flask and a web interface allows real-time predictions through a browser.

9. Conclusion

This project demonstrates how machine learning models can be deployed using Flask and made accessible through a simple web interface. It provides an easy way for users to interact with complex predictive models without requiring technical knowledge.